**Course Instructors**
**Assoc. Prof. Mervat Abuelkheir**
**Dr. Mary Mohsen**




MEDIA ENGINEERING AND TECHNOLOGY
Databases I · Winter 2026

**MILESTONE 2**

*Public Tests Guide*

# Local Business Delivery and Fulfillment Management System

**SUBMISSION**

**19/11/2026 (11:59 PM)**

# Table of Contents

**1. What You Get**

**2. Way 1: Run the Grader**

2.1 One-time setup

2.2 Run it

2.3 Read the result

2.4 Common messages

**3. Way 2: Expected Input and Output**

3.1 Before you start

3.2 How to read the tables

4.2 Basic Data Retrieval

4.3 Accounts: Registration, Login and Password

4.4 Admin

4.5 Partner

4.6 Customer

4.7 Rider

4.8 System-Level Requirements

# 1. What You Get

These files help you check your Milestone 2 submission before you submit it:

| File | What it is | Do I open it? |
|---|---|---|
| grader.py | The public test grader. It runs your Team_XX.sql file and tells you which tests passed and which failed. | No, you run it (Way 1). |
| public_tests.sql | The public tests used by the grader (109 tests). | No. Keep it next to grader.py. |
| seed_data.sql | The sample data that every test starts from (zones, users, businesses, orders, ...). | Only for Way 2. |
| This guide | How to use the grader, and the expected input and output of every public test. | Yes. |

> **Two ways to check your work.** Way 1 runs every public test for you in about one minute and prints PASS or FAIL. Way 2 shows, for every requirement, what goes in and what should come out, so you can try it yourself in Visual Studio.

# 2. Way 1: Run the Grader

## 2.1 One-time setup

1. Install **Python 3** from python.org. In the installer, tick **"Add python.exe to PATH"**.
2. Open **Command Prompt** and run: `pip install pyodbc`
3. Make sure **SQL Server 2019 or later** is installed and running (2019 is recommended). Its ODBC driver is installed with it.

## 2.2 Run it

1. Put **grader.py**, **public_tests.sql** and your **Team_XX.sql** in the same folder.
2. In Visual Studio, close the connection to your database (the grader needs to rebuild it).
3. Open the folder in File Explorer, type **cmd** in the address bar and press Enter.
4. Run: `python grader.py Team_10.sql` (use your own file name).
5. Wait about one minute and read the result.

> **Server not found?** Give the server name exactly as Visual Studio shows it in SQL Server Object Explorer:
> `python grader.py Team_10.sql --server "(localdb)\MSSQLLocalDB"`
> **Important:** the grader deletes your database LocalDeliveryMarketplace_Team_XX and builds it again from your file, exactly like a fresh server. Your .sql file is not changed.

## 2.3 Read the result

The grader first checks that your file runs without errors, then prints one line per test:

```
Running Team_10.sql ...
Your file ran without errors.
Running the public tests (about one minute) ...

PU-4.4.f-1  PASS  Create_Zone creates a zone
PU-4.4.f-2  FAIL  Create_Zone returns -2 for a duplicate name, -3 for a Zone Admin ...
                  - duplicate: expected return code -2 but got 0
PU-4.4.g-1  PASS  Update_Zone_Fee changes the fee without touching existing SubOrders
...
PASSED 108 of 109 public tests.
Full details of every failed test: C:\...\Team_10_results.txt
```

| You see | It means |
|---|---|
| PASS | Your requirement behaves as expected in that test. |
| FAIL | The lines under it say what was expected and what your code did. |
| PU-4.4.f-2 | Test 2 of requirement 4.4.f. Find it in Way 2 of this guide to see its input and expected output. |
| PP-1 ... PP-4 | Path tests: several requirements used one after the other, like a real user would. |
| Team_XX_results.txt | The full report, saved next to your .sql file. |

*Run the grader as often as you like: every test starts again from the seed data. Passing every public test does not guarantee full marks; the TAs also run private tests.*

## 2.4 Common messages

| Message | What to do |
|---|---|
| Your file raised N error(s) | Your file must run from top to bottom without errors. Fix the line shown. |
| FIX THIS FIRST: the seed data does not fit your tables | A table or column name, order or type differs from Section 3 and Appendix A. Fix the tables first; most tests fail until then. |
| the call raised an error instead of returning -3 | Return the error code with RETURN. Do not raise an error. |
| expected return code -4 but got -1 | You returned another error code. Check Section 2.4 and the table of the test in Way 2. |
| the rejected call changed the database | You returned an error code but already changed some rows. Check everything before you insert, update or delete. |
| expected columns (...) but got (...) | The output columns of your view or function have other names or another order. |
| expected 3 row(s), got 2. Missing rows: ... | Your output is different. Compare it with the Expected output table in Way 2. |
| Could not connect to SQL Server | Start SQL Server, or use --server with the name shown in Visual Studio. |
| pyodbc is not installed | Run  pip install pyodbc  in Command Prompt. |

# 3. Way 2: Expected Input and Output

## 3.1 Before you start

1. Run your **Team_XX.sql** file in Visual Studio.
2. Empty your tables: `EXEC clearAllTables;`
3. Run **seed_data.sql** (replace XX in its USE line with your team number).
4. Call your procedure or function with the values of one case and compare with the tables below.
5. Load the seed data again (steps 2 and 3) before the next test, because every test starts from the seed data.

## 3.2 How to read the tables

Every requirement has a Type line and one or more tests. The test ID matches the grader output.

| Part | What it shows |
|---|---|
| Input / Case 1, Case 2, ... | One column per call. Each row is a parameter, in the order of the description. NULL means you pass NULL. |
| Returns | The value your procedure returns. 0 is success (green); a negative number is an error code (red). For a function it is the value the function returns. |
| Tables after the call | "No change" means every table must stay exactly as it was. For a successful call, the "After case" table lists the table, the row and the column, and the value it must have. |
| Expected output | For views and table-valued functions: the exact column names in order, and the rows you must return (in any order). |

| Error code | Meaning (Section 2.4) |
|---|---|
| 0 | Success (stored procedures only) |
| -1 | Invalid input: a value breaks a constraint or a business rule |
| -2 | Duplicate: the value or record already exists |
| -3 | Not allowed: wrong user type, out of scope, not the owner, or Blacklisted |
| -4 | Wrong status: the current status does not allow the action |
| -5 | Zone mismatch: the active address is not in the zone of the business |
| -6 | Limit exceeded (for example more than 10 of one product in the cart) |
| -7 | Payment cannot be made |
| -8 | No available rider |
| -9 | Nothing to process (for example an empty cart or nothing to copy) |

> **Section 4.1:** createAllTables must create the 25 tables with the exact names, column order and types of Section 3 and Appendix A. After **clearAllTables** every table is empty and the next zone you insert gets zone_id 1 and the next user gets user_id 1.

## 4.2 Basic Data Retrieval

### a) allBusinessProfiles

**Type:** View **Requirement:** 4.2.a

**PU-4.2.a** allBusinessProfiles lists every business (any status) with type, zone and partner

**Expected output** (8 rows, in any order)

| business_id | business_name | business_type | zone_name | partner_name | status | opening_hour | closing_hour |
|---|---|---|---|---|---|---|---|
| 1 | Koshary El Tahrir | Restaurant | Maadi | Tarek Fouad | Active | 00:00 | 00:00 |
| 2 | Pizza Roma | Restaurant | Maadi | Laura Bianchi | Active | 00:00 | 00:00 |
| 3 | Seif Pharmacy | Pharmacy | Maadi | Seif Mahmoud | Active | 00:00 | 00:00 |
| 4 | Sushi Zen | Restaurant | Zamalek | Kenji Sato | Active | 00:00 | 00:00 |
| 5 | Zamalek Pharmacy | Pharmacy | Zamalek | Dina Fathy | Pending | 09:00 | 21:00 |
| 6 | Sweet Corner | Restaurant | New Cairo | Rania Ezz | Suspended | 10:00 | 02:00 |
| 7 | Cairo Grill | Restaurant | New Cairo | Hassan Gaber | Active | 00:00 | 00:00 |
| 8 | Nile Sweets | Restaurant | New Cairo | Nabil Fares | Rejected | 10:00 | 23:00 |

### b) BusinessesPerZone

**Type:** View **Requirement:** 4.2.b

**PU-4.2.b** BusinessesPerZone counts Active businesses and keeps empty zones

**Expected output** (4 rows, in any order)

| zone_id | zone_name | business_count |
|---|---|---|
| 1 | Maadi | 3 |
| 2 | Zamalek | 1 |
| 3 | New Cairo | 1 |
| 4 | Heliopolis | 0 |

### c) BusinessRatingsSummary

**Type:** View **Requirement:** 4.2.c

**PU-4.2.c** BusinessRatingsSummary averages feedback per business

**Expected output** (8 rows, in any order)

| business_id | business_name | avg_rating | ratings_count |
|---|---|---|---|
| 1 | Koshary El Tahrir | 3.50 | 2 |
| 2 | Pizza Roma | NULL | 0 |
| 3 | Seif Pharmacy | 4.00 | 1 |
| 4 | Sushi Zen | 5.00 | 1 |
| 5 | Zamalek Pharmacy | NULL | 0 |
| 6 | Sweet Corner | NULL | 0 |
| 7 | Cairo Grill | 4.00 | 1 |
| 8 | Nile Sweets | NULL | 0 |

## d) ActiveVouchers

**Type:** View **Requirement:** 4.2.d

**PU-4.2.d** ActiveVouchers returns Unused, non-expired vouchers

**Expected output** (3 rows, in any order)

| voucher_id | promo_code | customer_id | is_percentage | amount | expiry_date |
|---|---|---|---|---|---|
| 3 | WELCOME50 | 17 | 0 | 50.00 | 2030-12-31 |
| 4 | SAVE10 | 17 | 1 | 10.00 | 2030-12-31 |
| 7 | SARA25 | 18 | 1 | 25.00 | 2030-12-31 |

## e) PendingBusinessApprovals

**Type:** View **Requirement:** 4.2.e

**PU-4.2.e** PendingBusinessApprovals returns Pending businesses only

**Expected output** (1 row, in any order)

| business_id | business_name | business_type | zone_name | partner_name |
|---|---|---|---|---|
| 5 | Zamalek Pharmacy | Pharmacy | Zamalek | Dina Fathy |

## f) RiderPerformanceSummary

**Type:** View **Requirement:** 4.2.f

**PU-4.2.f** RiderPerformanceSummary lists every rider with completed deliveries and rating

**Expected output** (5 rows, in any order)

| rider_id | rider_name | zone_name | completed_deliveries | avg_rating |
|---|---|---|---|---|
| 12 | Ali Ibrahim | Maadi | 2 | 4.50 |
| 13 | Mostafa Kamal | Maadi | 2 | 4.00 |
| 14 | Yehia Saad | Maadi | 0 | NULL |
| 15 | Ziad Nabil | Zamalek | 1 | 5.00 |
| 16 | Fady Ramzy | New Cairo | 1 | 3.00 |

## g) TopSellingProducts

**Type:** View **Requirement:** 4.2.g

**PU-4.2.g** TopSellingProducts returns the top 10 by units, ignoring cancelled SubOrders

**Expected output** (10 rows, in any order)

| product_id | product_name | business_name | business_type | units_sold | total_revenue |
|---|---|---|---|---|---|
| 2 | Koshary Small | Koshary El Tahrir | Restaurant | 3 | 120.00 |
| 1 | Koshary Large | Koshary El Tahrir | Restaurant | 2 | 120.00 |
| 3 | Rice Pudding | Koshary El Tahrir | Restaurant | 2 | 40.00 |
| 8 | Panadol Extra | Seif Pharmacy | Pharmacy | 2 | 90.00 |
| 12 | Miso Soup | Sushi Zen | Restaurant | 2 | 100.00 |
| 5 | Margherita Pizza | Pizza Roma | Restaurant | 1 | 100.00 |
| 6 | Pepperoni Pizza | Pizza Roma | Restaurant | 1 | 150.00 |
| 9 | Vitamin C | Seif Pharmacy | Pharmacy | 1 | 80.00 |
| 10 | Hand Sanitizer | Seif Pharmacy | Pharmacy | 1 | 30.00 |
| 11 | Salmon Roll | Sushi Zen | Restaurant | 1 | 180.00 |

## h) ZoneDeliveryPerformance

**Type:** View **Requirement:** 4.2.h

**PU-4.2.h** ZoneDeliveryPerformance counts Delivered deliveries per zone

**Expected output** (4 rows, in any order)

| zone_id | zone_name | delivery_count | avg_tracking_time |
|---|---|---|---|
| 1 | Maadi | 4 | 22.50 |
| 2 | Zamalek | 1 | 35.00 |
| 3 | New Cairo | 1 | 40.00 |
| 4 | Heliopolis | 0 | NULL |

## 4.3 Accounts: Registration, Login and Password

### a) Customer_Register

**Type:** Stored Procedure **Requirement:** 4.3.a

**PU-4.3.a-1** Customer_Register creates an Active customer with a 0 wallet and a phone

| Input | Case 1 |
|---|---|
| first_name | Mariam |
| middle_name | *NULL* |
| last_name | Wael |
| username | mariam.wael |
| password | Pass@025 |
| email | mariam.wael@mail.com |
| phone | 01000000025 |
| dob | 2001-05-05 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Users | new user mariam.wael | account_status | Active |
| Customer | new user mariam.wael | wallet_balance | 0.00 |
| User_Phone | new user mariam.wael | phone | 01000000025 |

**PU-4.3.a-2** Customer_Register returns -2 for a duplicate username and -1 for an invalid phone

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| first_name | Mariam | Mariam |
| middle_name | *NULL* | *NULL* |
| last_name | Wael | Wael |
| username | ahmed.salem | mariam.wael |
| password | Pass@025 | Pass@025 |
| email | mariam.wael@mail.com | mariam.wael@mail.com |
| phone | 01000000025 | 0100 |
| dob | 2001-05-05 | 2001-05-05 |
| **Returns** | **-2 (duplicate)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change |

### b) Partner_Register

**Type:** Stored Procedure **Requirement:** 4.3.b

**PU-4.3.b-1** Partner_Register creates a partner without a business

| Input | Case 1 |
|---|---|
| first_name | Omar |
| middle_name | *NULL* |
| last_name | Fawzy |

| Input | Case 1 |
|---|---|
| username | omar.fawzy |
| password | Pass@025 |
| email | omar.fawzy@mail.com |
| phone | 01000000025 |
| dob | 1988-08-08 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Partner | new user omar.fawzy | partner_id | the new user_id (a Partner row exists) |
| Business | partner omar.fawzy | (any row) | none: the business is registered later |

**PU-4.3.b-2** Partner_Register returns -2 for a duplicate email

| Input | Case 1 |
|---|---|
| first_name | Omar |
| middle_name | *NULL* |
| last_name | Fawzy |
| username | omar.fawzy |
| password | Pass@025 |
| email | tarek.fouad@mail.com |
| phone | 01000000025 |
| dob | 1988-08-08 |
| **Returns** | **-2 (duplicate)** |
| **Tables after the call** | No change |

## c) Rider_Register

**Type:** Stored Procedure **Requirement:** 4.3.c

**PU-4.3.c-1** Rider_Register creates an available rider in the given zone

| Input | Case 1 |
|---|---|
| first_name | Hady |
| middle_name | *NULL* |
| last_name | Wagdy |
| username | hady.wagdy |
| password | Pass@025 |
| email | hady.wagdy@mail.com |
| phone | 01000000025 |
| dob | 1997-03-03 |
| vehicle_type | Motorcycle |
| plate_no | MAA-9999 |
| license_no | LIC-9999 |
| zone_id | 2 |

| Input | Case 1 |
|---|---|
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Rider | new user hady.wagdy | vehicle_type | Motorcycle |
| Rider | new user hady.wagdy | plate_no | MAA-9999 |
| Rider | new user hady.wagdy | license_no | LIC-9999 |
| Rider | new user hady.wagdy | is_available | 1 |
| Rider | new user hady.wagdy | zone_id | 2 |

**PU-4.3.c-2** Rider_Register returns -2 for a duplicate plate number

| Input | Case 1 |
|---|---|
| `first_name` | Hady |
| `middle_name` | *NULL* |
| `last_name` | Wagdy |
| `username` | hady.wagdy |
| `password` | Pass@025 |
| `email` | hady.wagdy@mail.com |
| `phone` | 01000000025 |
| `dob` | 1997-03-03 |
| `vehicle_type` | Motorcycle |
| `plate_no` | MAA-1201 |
| `license_no` | LIC-9999 |
| `zone_id` | 2 |
| **Returns** | **-2 (duplicate)** |
| **Tables after the call** | No change |

## d) CustomerLoginValidation

**Type:** Function (scalar) **Requirement:** 4.3.d

**PU-4.3.d** CustomerLoginValidation accepts valid Active customers only

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| `username` | ahmed.salem | ahmed.salem | tarek.fouad | youssef.amin |
| `password` | Pass@017 | wrong | Pass@004 | Pass@021 |
| **Returns** | **1** | **0** | **0** | **0** |

## e) PartnerLoginValidation

**Type:** Function (scalar) **Requirement:** 4.3.e

**PU-4.3.e** PartnerLoginValidation accepts partners only

| Input | Case 1 | Case 2 |
|---|---|---|
| `username` | tarek.fouad | ahmed.salem |
| `password` | Pass@004 | Pass@017 |
| **Returns** | **1** | **0** |

## f) RiderLoginValidation

**Type:** Function (scalar) **Requirement:** 4.3.f

**PU-4.3.f** RiderLoginValidation accepts riders only

| Input | Case 1 | Case 2 |
|---|---|---|
| username | ali.ibrahim | ahmed.salem |
| password | Pass@012 | Pass@017 |
| **Returns** | **1** | **0** |

## g) AdminLoginValidation

**Type:** Function (scalar) **Requirement:** 4.3.g

**PU-4.3.g** AdminLoginValidation accepts admins only

| Input | Case 1 | Case 2 |
|---|---|---|
| username | mona.hassan | ahmed.salem |
| password | Pass@001 | Pass@017 |
| **Returns** | **1** | **0** |

## h) Change_Password

**Type:** Stored Procedure **Requirement:** 4.3.h

**PU-4.3.h-1** Change_Password changes the password and sets success = 1

| Input | Case 1 |
|---|---|
| user_id | 17 |
| old_password | Pass@017 |
| new_password | New@123 |
| new_password_repeat | New@123 |
| success | *(output)* |
| **Returns** | **0, success = 1** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Users | user_id 17 | password | New@123 (CustomerLoginValidation(ahmed.salem, New@123) returns 1) |

**PU-4.3.h-2** Change_Password returns -1 with success = 0 and changes nothing when the repeat differs or the old password is wrong

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| user_id | 17 | 17 |
| old_password | Pass@017 | wrong |
| new_password | New@123 | New@123 |
| new_password_repeat | Other@123 | New@123 |
| success | *(output)* | *(output)* |

| Input | Case 1 | Case 2 |
| --- | --- | --- |
| **Returns** | **-1 (invalid input), success = 0** | **-1 (invalid input), success = 0** |
| **Tables after the call** | No change | No change |

## 4.4 Admin

### a) Create_Admin

**Type:** Stored Procedure **Requirement:** 4.4.a

**PU-4.4.a-1** Create_Admin: a Super Admin creates a Zone Admin

| Input | Case 1 |
|---|---|
| first_name | Laila |
| middle_name | *NULL* |
| last_name | Gamal |
| username | laila.gamal |
| password | Pass@028 |
| email | laila.gamal@mail.com |
| phone | 01000000028 |
| dob | 1989-09-09 |
| role | Zone Admin |
| zone_id | 4 |
| super_admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Admin | new user laila.gamal | role | Zone Admin |
| Admin | new user laila.gamal | zone_id | 4 |

**PU-4.4.a-2** Create_Admin returns -3 when a Zone Admin calls it

| Input | Case 1 |
|---|---|
| first_name | Laila |
| middle_name | *NULL* |
| last_name | Gamal |
| username | laila.gamal |
| password | Pass@028 |
| email | laila.gamal@mail.com |
| phone | 01000000028 |
| dob | 1989-09-09 |
| role | Zone Admin |
| zone_id | 4 |
| super_admin_id | 2 |
| **Returns** | **-3 (not allowed)** |
| **Tables after the call** | No change |

## b) Approve_Business

**Type:** Stored Procedure **Requirement:** 4.4.b

**PU-4.4.b-1** Approve_Business: the Zone Admin of the zone approves a Pending business

| Input | Case 1 |
|---|---|
| business_id | 5 |
| admin_id | 3 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | business_id 5 | status | Active |

**PU-4.4.b-2** Approve_Business returns -3 for a Zone Admin of another zone and -4 for a business that is not Pending

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 5 | 1 |
| admin_id | 2 | 1 |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change |

## c) Reject_Business

**Type:** Stored Procedure **Requirement:** 4.4.c

**PU-4.4.c-1** Reject_Business changes a Pending business to Rejected

| Input | Case 1 |
|---|---|
| business_id | 5 |
| admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | business_id 5 | status | Rejected |

**PU-4.4.c-2** Reject_Business returns -3 for a Zone Admin of another zone and -4 for a business that is not Pending

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 5 | 2 |
| admin_id | 2 | 1 |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change |

## d) Suspend_Business

**Type:** Stored Procedure **Requirement:** 4.4.d

**PU-4.4.d-1** Suspend_Business hides the business, cancels its Placed/Assigned SubOrders and refunds them

| Input | Case 1 |
|---|---|
| business_id | 3 |
| admin_id | 2 |
| reason | Expired medicines |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | business_id 3 | status | Suspended |
| Business | business_id 3 | suspension_reason | Expired medicines |
| Product | every product of business 3 | is_available | 0 |
| SubOrder | suborder_id 3 | status | Delivered (unchanged) |
| SubOrder | suborder_id 10 | status | Cancelled |
| SubOrder | suborder_id 11 | status | Cancelled |
| Orders | order_id 7 | status | Cancelled |
| Orders | order_id 6 | status | In Progress |
| Payment | order_id 7 | refunded_amount | 100.00 |
| Customer | customer_id 18 | wallet_balance | 50.00 (unchanged: order 7 was Cash) |
| Payment | order_id 6 | refunded_amount | 50.00 (30.00 + 20.00 fee) |
| Customer | customer_id 17 | wallet_balance | 550.00 |

**PU-4.4.d-2** Suspend_Business restores the voucher of an order it cancels entirely and caps the refund at the payment

| Input | Case 1 |
|---|---|
| business_id | 2 |
| admin_id | 1 |
| reason | Health inspection |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Orders | order_id 8 | status | Cancelled |
| Orders | order_id 8 | voucher_id | NULL |
| Voucher | voucher_id 5 | status | Unused |
| Payment | order_id 8 | refunded_amount | 155.00 (capped at the payment) |
| Customer | customer_id 22 | wallet_balance | 180.00 |

**PU-4.4.d-3** Suspend_Business returns -3 out of scope and -4 for a business that is not Active

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 3 | 5 |

| Input | Case 1 | Case 2 |
|---|---|---|
| admin_id | 3 | 3 |
| reason | Reason | Reason |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change |

## e) Reinstate_Business

**Type:** Stored Procedure **Requirement:** 4.4.e

**PU-4.4.e-1** Reinstate_Business: a Super Admin reinstates a Suspended business, products stay unavailable

| Input | Case 1 |
|---|---|
| business_id | 6 |
| super_admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | business_id 6 | status | Active |
| Business | business_id 6 | suspension_reason | NULL |
| Product | product_id 15 | is_available | 0 (stays unavailable) |

**PU-4.4.e-2** Reinstate_Business returns -3 for a Zone Admin and -4 for a business that is not Suspended

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 6 | 1 |
| super_admin_id | 2 | 1 |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change |

## f) Create_Zone

**Type:** Stored Procedure **Requirement:** 4.4.f

**PU-4.4.f-1** Create_Zone creates a zone

| Input | Case 1 |
|---|---|
| zone_name | Dokki |
| delivery_fee | 18.00 |
| super_admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Zone | new zone Dokki | delivery_fee | 18.00 |

**PU-4.4.f-2** Create_Zone returns -2 for a duplicate name, -3 for a Zone Admin and -1 for a negative fee

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| zone_name | Maadi | Dokki | Dokki |
| delivery_fee | 18.00 | 18.00 | -1.00 |
| super_admin_id | 1 | 2 | 1 |
| **Returns** | **-2 (duplicate)** | **-3 (not allowed)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change | No change |

## g) Update_Zone_Fee

**Type:** Stored Procedure **Requirement:** 4.4.g

**PU-4.4.g-1** Update_Zone_Fee changes the fee without touching existing SubOrders

| Input | Case 1 |
|---|---|
| zone_id | 1 |
| new_fee | 22.50 |
| super_admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Zone | zone_id 1 | delivery_fee | 22.50 |
| SubOrder | suborder_id 1 | delivery_fee | 20.00 (unchanged) |

**PU-4.4.g-2** Update_Zone_Fee returns -3 for a Zone Admin and -1 for a negative fee

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| zone_id | 1 | 1 |
| new_fee | 22.50 | -5.00 |
| super_admin_id | 2 | 1 |
| **Returns** | **-3 (not allowed)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change |

## h) Assign_Admin_Zone

**Type:** Stored Procedure **Requirement:** 4.4.h

**PU-4.4.h-1** Assign_Admin_Zone moves a Zone Admin

| Input | Case 1 |
|---|---|
| admin_id | 2 |
| zone_id | 4 |
| super_admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Admin | admin_id 2 | zone_id | 4 |

**PU-4.4.h-2** Assign_Admin_Zone returns -1 for a Super Admin target and -3 for a Zone Admin caller

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| admin_id | 1 | 3 |
| zone_id | 4 | 4 |
| super_admin_id | 1 | 2 |
| **Returns** | **-1 (invalid input)** | **-3 (not allowed)** |
| **Tables after the call** | No change | No change |

## i) Issue_Voucher

**Type:** Stored Procedure **Requirement:** 4.4.i

**PU-4.4.i-1** Issue_Voucher creates an Unused voucher

| Input | Case 1 |
|---|---|
| customer_id | 17 |
| promo_code | NEW20 |
| is_percentage | 0 |
| amount | 20.00 |
| expiry_date | 2030-12-31 |
| admin_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Voucher | promo_code NEW20 | status | Unused |
| Voucher | promo_code NEW20 | amount | 20.00 |
| Voucher | promo_code NEW20 | is_percentage | 0 |
| Voucher | promo_code NEW20 | customer_id | 17 |

**PU-4.4.i-2** Issue_Voucher returns -3 for a Blacklisted customer and -2 for a duplicate promo code

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| customer_id | 21 | 17 |
| promo_code | NEW20 | SAVE10 |
| is_percentage | 0 | 0 |
| amount | 20.00 | 20.00 |
| expiry_date | 2030-12-31 | 2030-12-31 |
| admin_id | 1 | 1 |
| **Returns** | **-3 (not allowed)** | **-2 (duplicate)** |
| **Tables after the call** | No change | No change |

## j) Blacklist_Customer

**Type:** Stored Procedure **Requirement:** 4.4.j

**PU-4.4.j-1** Blacklist_Customer blocks the account, cancels cancellable orders, refunds and deletes Unused vouchers

| Input | Case 1 |
|---|---|
| customer_id | 22 |
| admin_id | 2 |
| reason | Chargeback fraud |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Users | user_id 22 | account_status | Blacklisted |
| Blacklist | customer_id 22 | admin_id | 2 |
| Blacklist | customer_id 22 | reason | Chargeback fraud |
| Orders | order_id 8 | status | Cancelled |
| Payment | order_id 8 | refunded_amount | 155.00 |
| Customer | customer_id 22 | wallet_balance | 180.00 |
| Voucher | vouchers of customer 22 | (rows) | none left |

**PU-4.4.j-2** Blacklist_Customer returns -4 if already Blacklisted and -3 for a Zone Admin without an address in the zone

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| customer_id | 21 | 22 |
| admin_id | 1 | 3 |
| reason | Again | Fraud |
| **Returns** | **-4 (wrong status)** | **-3 (not allowed)** |
| **Tables after the call** | No change | No change |

## k) Lift_Customer_Blacklist

**Type:** Stored Procedure **Requirement:** 4.4.k

**PU-4.4.k-1** Lift_Customer_Blacklist: the admin who blacklisted the customer lifts it

| Input | Case 1 |
|---|---|
| customer_id | 21 |
| admin_id | 2 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Users | user_id 21 | account_status | Active |
| Blacklist | customer_id 21 | (row) | removed |

**PU-4.4.k-2** Lift_Customer_Blacklist returns -3 for any other admin, even a Super Admin

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| `customer_id` | 21 | 21 |
| `admin_id` | 1 | 3 |
| **Returns** | **-3 (not allowed)** | **-3 (not allowed)** |
| **Tables after the call** | No change | No change |

## l) ZoneOrderReport

**Type:** Table-Valued Function **Requirement:** 4.4.l

**PU-4.4.l** ZoneOrderReport for Maadi and New Cairo in September

**Input**

| zone_id | from_date | to_date |
|---|---|---|
| 1 | 2026-09-01 | 2026-09-30 |

**Expected output** (1 row, in any order)

| zone_id | zone_name | number_of_orders | avg_order_total | platform_commission |
|---|---|---|---|---|
| 1 | Maadi | 5 | 169.00 | 73.00 |

**Input**

| zone_id | from_date | to_date |
|---|---|---|
| 3 | 2026-09-01 | 2026-09-30 |

**Expected output** (1 row, in any order)

| zone_id | zone_name | number_of_orders | avg_order_total | platform_commission |
|---|---|---|---|---|
| 3 | New Cairo | 1 | 350.00 | 22.00 |

## m) TopRiders

**Type:** Table-Valued Function **Requirement:** 4.4.m

**PU-4.4.m** TopRiders ranks riders by deliveries delivered in the period

**Input**

| from_date | to_date |
|---|---|
| 2026-09-01 | 2026-09-30 |

**Expected output** (4 rows, in any order)

| rider_rank | rider_id | rider_name | completed_delivery_count |
|---|---|---|---|
| 1 | 12 | Ali Ibrahim | 2 |
| 2 | 13 | Mostafa Kamal | 2 |
| 3 | 15 | Ziad Nabil | 1 |
| 4 | 16 | Fady Ramzy | 1 |

## n) Daily_Revenue_Report

**Type:** Table-Valued Function **Requirement:** 4.4.n

**PU-4.4.n** Daily_Revenue_Report splits a day by zone and business type

**Input**

| report_date |
|---|
| 2026-09-02 |

**Expected output** (2 rows, in any order)

| zone_name | business_type | number_of_suborders | gross_sales | delivery_fees | platform_commission |
|---|---|---|---|---|---|
| Maadi | Pharmacy | 1 | 90.00 | 20.00 | 9.00 |
| Maadi | Restaurant | 1 | 40.00 | 20.00 | 4.00 |

## 4.5 Partner

### a) Register_Business

**Type:** Stored Procedure **Requirement:** 4.5.a

**PU-4.5.a-1** Register_Business creates a Pending restaurant with its phone and subtype row

| Input | Case 1 |
|---|---|
| partner_id | 11 |
| name | Mai Bakery |
| description | Fresh bread |
| link_website | *NULL* |
| zone_id | 1 |
| opening_hour | 08:00 |
| closing_hour | 23:00 |
| expected_delivery_time | 25 |
| phone | 02200000011 |
| business_type | Restaurant |
| cuisine_type | Egyptian |
| food_category | Bakery |
| pharmacy_type | *NULL* |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | partner_id 11 | name | Mai Bakery |
| Business | partner_id 11 | status | Pending |
| Business | partner_id 11 | zone_id | 1 |
| Business | partner_id 11 | expected_delivery_time | 25 |
| Business_Phone | that business | phone | 02200000011 |
| Restaurant | that business | cuisine_type | Egyptian |
| Restaurant | that business | food_category | Bakery |

**PU-4.5.a-2** Register_Business returns -2 for a partner who already owns a business and -1 for a Restaurant without food_category

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| partner_id | 4 | 11 |
| name | Second | Mai Bakery |
| description | *NULL* | *NULL* |
| link_website | *NULL* | *NULL* |
| zone_id | 1 | 1 |
| opening_hour | 08:00 | 08:00 |
| closing_hour | 23:00 | 23:00 |
| expected_delivery_time | 25 | 25 |

| Input | Case 1 | Case 2 |
|---|---|---|
| phone | 02200000011 | 02200000011 |
| business_type | Restaurant | Restaurant |
| cuisine_type | Egyptian | Egyptian |
| food_category | Bakery | *NULL* |
| pharmacy_type | *NULL* | *NULL* |
| **Returns** | **-2 (duplicate)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change |

**PU-4.5.a-3** Register_Business re-registers a Rejected business: same row, new data, back to Pending

| Input | Case 1 |
|---|---|
| partner_id | 24 |
| name | Nile Pharmacy |
| description | Now a pharmacy |
| link_website | *NULL* |
| zone_id | 3 |
| opening_hour | 09:00 |
| closing_hour | 22:00 |
| expected_delivery_time | 30 |
| phone | 02200000088 |
| business_type | Pharmacy |
| cuisine_type | *NULL* |
| food_category | *NULL* |
| pharmacy_type | Retail |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | partner_id 24 | (rows) | still 1 row, business_id 8 |
| Business | business_id 8 | name | Nile Pharmacy |
| Business | business_id 8 | status | Pending |
| Business_Phone | business_id 8 | phone | only 02200000088 |
| Restaurant | business_id 8 | (row) | removed |
| Pharmacy | business_id 8 | pharmacy_type | Retail |

## b) Update_Business_Info

**Type:** Stored Procedure **Requirement:** 4.5.b

**PU-4.5.b-1** Update_Business_Info updates hours and website and adds a phone

| Input | Case 1 |
|---|---|
| business_id | 1 |
| opening_hour | 08:00 |
| closing_hour | 22:00 |
| phone | 02200000099 |

| Input | Case 1 |
|---|---|
| link_website | www.new.example |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Business | business_id 1 | opening_hour | 08:00 |
| Business | business_id 1 | closing_hour | 22:00 |
| Business | business_id 1 | link_website | www.new.example |
| Business_Phone | business_id 1 | (rows) | 3 phones (02200000099 added) |

**PU-4.5.b-2** Update_Business_Info returns -1 for a malformed phone and -2 for a phone the business already has

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 1 | 1 |
| opening_hour | 08:00 | 08:00 |
| closing_hour | 22:00 | 22:00 |
| phone | abc | 02200000001 |
| link_website | *NULL* | *NULL* |
| **Returns** | **-1 (invalid input)** | **-2 (duplicate)** |
| **Tables after the call** | No change | No change |

## c) Upsert_Product

**Type:** Stored Procedure **Requirement:** 4.5.c

**PU-4.5.c-1** Upsert_Product inserts a new product when product_id is NULL and updates an existing one

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| business_id | 1 | 1 |
| product_id | *NULL* | 1 |
| name | Molokhia | Koshary Large |
| description | Green soup | Large box |
| category | Main Dishes | Main Dishes |
| price | 55.00 | 65.00 |
| is_available | 1 | 1 |
| **Returns** | **0** | **0** |
| **Tables after the call** | See "After case 1" | See "After case 2" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Product | new product_id 21 | name | Molokhia |
| Product | product_id 21 | price | 55.00 |
| Product | product_id 21 | discount_amount | 0.00 |
| Product | product_id 21 | is_available | 1 |

**After case 2** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Product | product_id 1 | price | 65.00 |
| CartItem | cart_id 1, product_id 1 | unit_price | 60.00 (placed orders keep their price) |

**PU-4.5.c-2** Upsert_Product returns -3 for a product of another business, -4 for a Suspended business and -1 for invalid data

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| business_id | 1 | 6 | 1 | 2 |
| product_id | 5 | *NULL* | *NULL* | 5 |
| name | Margherita Pizza | New Sweet | Free | Margherita Pizza |
| description | *NULL* | *NULL* | *NULL* | *NULL* |
| category | Pizza | Desserts | Main Dishes | Pizza |
| price | 120.00 | 30.00 | 0.00 | 30.00 |
| is_available | 1 | 1 | 1 | 1 |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** | **-1 (invalid input)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change | No change | No change |

## d) Toggle_Product_Status

**Type:** Stored Procedure **Requirement:** 4.5.d

**PU-4.5.d-1** Toggle_Product_Status deactivates a product

| Input | Case 1 |
|---|---|
| product_id | 1 |
| is_available | 0 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Product | product_id 1 | is_available | 0 |

**PU-4.5.d-2** Toggle_Product_Status returns -4 when reactivating while the business is Suspended

| Input | Case 1 |
|---|---|
| product_id | 15 |
| is_available | 1 |
| **Returns** | **-4 (wrong status)** |
| **Tables after the call** | No change |

## e) Set_Product_Promotion

**Type:** Stored Procedure **Requirement:** 4.5.e

**PU-4.5.e-1** Set_Product_Promotion sets a discount amount and 0 removes it

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| product_id | 1 | 1 |

| Input | Case 1 | Case 2 |
|---|---|---|
| `discount_amount` | 30.00 | 0 |
| **Returns** | **0** | **0** |
| **Tables after the call** | See "After case 1" | See "After case 2" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Product | product_id 1 | discount_amount | 30.00 |

**After case 2** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Product | product_id 1 | discount_amount | 0.00 |

**PU-4.5.e-2** Set_Product_Promotion returns -1 for a discount above 50% of the price or a negative one

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| `product_id` | 1 | 1 |
| `discount_amount` | 30.01 | -1.00 |
| **Returns** | **-1 (invalid input)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change |

## f) BusinessSalesReport

**Type:** Table-Valued Function **Requirement:** 4.5.f

**PU-4.5.f** BusinessSalesReport for Koshary El Tahrir in September

**Input**

| business_id | from_date | to_date |
|---|---|---|
| 1 | 2026-09-01 | 2026-09-30 |

**Expected output** (3 rows, in any order)

| product_id | product_name | units_sold | revenue_per_product |
|---|---|---|---|
| 1 | Koshary Large | 2 | 120.00 |
| 2 | Koshary Small | 3 | 120.00 |
| 3 | Rice Pudding | 2 | 40.00 |

## g) BusinessFeedbackOverview

**Type:** Table-Valued Function **Requirement:** 4.5.g

**PU-4.5.g** BusinessFeedbackOverview with star breakdown

**Input**

| business_id |
|---|
| 1 |

**Expected output** (1 row, in any order)

| avg_rating | number_of_feedbacks | five_star | four_star | three_star | two_star | one_star |
|---|---|---|---|---|---|---|
| 3.50 | 2 | 1 | 0 | 0 | 1 | 0 |

## 4.6 Customer

### a) Add_Customer_Address

**Type:** Stored Procedure **Requirement:** 4.6.a

**PU-4.6.a-1** Add_Customer_Address adds an address that becomes the only active one

| Input | Case 1 |
|---|---|
| customer_id | 17 |
| description | New Street 5 |
| zone_id | 1 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Address | new address_id 13 | is_active | 1 |
| Address | other addresses of customer 17 | is_active | 0 |
| Address | customer 17 | (rows) | 5 addresses |

**PU-4.6.a-2** Add_Customer_Address returns -1 for an empty description

| Input | Case 1 |
|---|---|
| customer_id | 17 |
| description | *(empty)* |
| zone_id | 1 |
| **Returns** | **-1 (invalid input)** |
| **Tables after the call** | No change |

### b) Update_Delete_Customer_Address

**Type:** Stored Procedure **Requirement:** 4.6.b

**PU-4.6.b-1** Update_Delete_Customer_Address updates and deletes the customer's own addresses

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| customer_id | 17 | 17 |
| address_id | 3 | 2 |
| description | Dokki New Flat | *NULL* |
| zone_id | 1 | *NULL* |
| action | Update | Delete |
| **Returns** | **0** | **0** |
| **Tables after the call** | See "After case 1" | See "After case 2" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Address | address_id 3 | description | Dokki New Flat |
| Address | address_id 3 | zone_id | 1 |

**After case 2** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Address | address_id 2 | (row) | deleted |

**PU-4.6.b-2** Update_Delete_Customer_Address returns -3 for another customer's address and -4 for an address of a Placed/In Progress order or used by a cart

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| `customer_id` | 17 | 17 | 17 |
| `address_id` | 4 | 1 | 3 |
| `description` | X | X | *NULL* |
| `zone_id` | 1 | 1 | *NULL* |
| `action` | Update | Update | Delete |
| **Returns** | **-3 (not allowed)** | **-4 (wrong status)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change | No change |

## c) Change_Active_Address

**Type:** Stored Procedure **Requirement:** 4.6.c

**PU-4.6.c-1** Change_Active_Address makes the selected address the only active one

| Input | Case 1 |
|---|---|
| `customer_id` | 17 |
| `new_address_id` | 2 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Address | address_id 2 | is_active | 1 |
| Address | other addresses of customer 17 | is_active | 0 |
| Cart | Active carts of customer 17 | (rows) | 2 (unchanged) |

**PU-4.6.c-2** Change_Active_Address returns -3 for an address that is not the customer's

| Input | Case 1 |
|---|---|
| `customer_id` | 17 |
| `new_address_id` | 4 |
| **Returns** | **-3 (not allowed)** |
| **Tables after the call** | No change |

## d) Add_Card

**Type:** Stored Procedure **Requirement:** 4.6.d

**PU-4.6.d-1** Add_Card saves a card

| Input | Case 1 |
|---|---|
| `customer_id` | 17 |
| `card_no` | 4999999999999999 |

| Input | Case 1 |
|---|---|
| card_holder_name | Ahmed Salem |
| expiration_date | 2031-01-31 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Card | customer_id 17 | card_no | 4999999999999999 (new row) |

**PU-4.6.d-2** Add_Card returns -1 for an expired card or a bad number and -2 for a card already saved

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| customer_id | 17 | 17 | 17 |
| card_no | 4999999999999999 | 4999 | 4111111111111111 |
| card_holder_name | Ahmed Salem | Ahmed Salem | Ahmed Salem |
| expiration_date | 2020-01-31 | 2031-01-31 | 2031-01-31 |
| **Returns** | **-1 (invalid input)** | **-1 (invalid input)** | **-2 (duplicate)** |
| **Tables after the call** | No change | No change | No change |

## e) Create_Cart

**Type:** Stored Procedure **Requirement:** 4.6.e

**PU-4.6.e-1** Create_Cart creates an Active cart for the active address

| Input | Case 1 |
|---|---|
| customer_id | 23 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Cart | new cart of customer 23 | status | Active |
| Cart | new cart of customer 23 | address_id | 11 (his active address) |

**PU-4.6.e-2** Create_Cart returns -2 when the active address already has an Active cart and -3 for a Blacklisted customer

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| customer_id | 17 | 21 |
| **Returns** | **-2 (duplicate)** | **-3 (not allowed)** |
| **Tables after the call** | No change | No change |

## f) Add_To_Cart

**Type:** Stored Procedure **Requirement:** 4.6.f

**PU-4.6.f-1** Add_To_Cart adds a new line and increases an existing line, replacing the note

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| cart_id | 9 | 9 | 9 |
| product_id | 2 | 1 | 1 |
| quantity | 2 | 3 | 1 |
| notes | *NULL* | Spicy | *NULL* |
| **Returns** | **0** | **0** | **0** |
| **Tables after the call** | See "After case 1" | See "After case 2" | See "After case 3" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| CartItem | cart_id 9, product_id 2 | quantity | 2 |
| CartItem | cart_id 9, product_id 2 | unit_price | NULL |

**After case 2** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| CartItem | cart_id 9, product_id 1 | quantity | 4 (1 + 3) |
| CartItem | cart_id 9, product_id 1 | notes | Spicy |

**After case 3** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| CartItem | cart_id 9, product_id 1 | quantity | 5 |
| CartItem | cart_id 9, product_id 1 | notes | Spicy (kept: no new note) |

**PU-4.6.f-2** Add_To_Cart returns -6 above 10, -5 outside the zone, -1 for quantity 0 and -4 for a cart that cannot be edited

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| cart_id | 9 | 9 | 9 | 1 |
| product_id | 1 | 11 | 2 | 2 |
| quantity | 10 | 1 | 0 | 1 |
| notes | *NULL* | *NULL* | *NULL* | *NULL* |
| **Returns** | **-6 (limit exceeded)** | **-5 (zone mismatch)** | **-1 (invalid input)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change | No change | No change |

| Input | Case 5 |
|---|---|
| cart_id | 15 |
| product_id | 11 |
| quantity | 1 |
| notes | *NULL* |
| **Returns** | **-4 (wrong status)** |
| **Tables after the call** | No change |

## g) Remove_From_Cart

**Type:** Stored Procedure **Requirement:** 4.6.g

**PU-4.6.g** Remove_From_Cart removes a line; -4 for a Checked Out cart

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| cart_id | 9 | 1 |
| product_id | 8 | 1 |
| **Returns** | **0** | **-4 (wrong status)** |
| **Tables after the call** | See "After case 1" | No change |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| CartItem | cart_id 9, product_id 8 | (row) | removed |

## h) Calculate_Voucher_Discount

**Type:** Function (scalar) **Requirement:** 4.6.h

**PU-4.6.h** Calculate_Voucher_Discount handles fixed, percentage, cap and invalid vouchers

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| voucher_id | 3 | 4 | 3 | 1 |
| order_subtotal | 200.00 | 200.00 | 30.00 | 100.00 |
| **Returns** | **50.00** | **20.00** | **30.00** | **0.00** |

| Input | Case 5 | Case 6 | Case 7 | Case 8 |
|---|---|---|---|---|
| voucher_id | 6 | *NULL* | 999 | 3 |
| order_subtotal | 100.00 | 100.00 | 100.00 | -5.00 |
| **Returns** | **0.00** | **0.00** | **0.00** | **-1.00** |

## i) Checkout_Cart

**Type:** Stored Procedure **Requirement:** 4.6.i

**PU-4.6.i-1** Checkout_Cart creates the order, SubOrders, Deliveries and Payment (voucher, wallet)

| Input | Case 1 |
|---|---|
| cart_id | 9 |
| voucher_id | 3 |
| payment_method | Wallet |
| card_id | *NULL* |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Orders | new order of cart 9 | status | Placed |
| Orders | new order of cart 9 | discount_amount | 50.00 |
| Orders | new order of cart 9 | total_amount | 140.00 (150.00 - 50.00 + 2 x 20.00) |

| Table | Row | Column | Expected value |
|---|---|---|---|
| Orders | new order of cart 9 | zone_id | 1 |
| Orders | new order of cart 9 | voucher_id | 3 |
| Orders | new order of cart 9 | order_date | now |
| SubOrder | business_id 1 | status / total_price / delivery_fee | Assigned / 60.00 / 20.00 |
| Delivery | of that SubOrder | rider_id | 12 |
| SubOrder | business_id 3 | status / total_price / delivery_fee | Assigned / 90.00 / 20.00 |
| Delivery | of that SubOrder | rider_id | 13 |
| Payment | new order | amount / currency / method | 140.00 / EGP / Wallet |
| Payment | new order | refunded_amount / card_id | 0.00 / NULL |
| Customer | customer_id 17 | wallet_balance | 360.00 |
| Voucher | voucher_id 3 | status | Used |
| Cart | cart_id 9 | status | Checked Out |
| CartItem | cart_id 9, product_id 8 | unit_price | 45.00 |

**PU-4.6.i-2** Checkout_Cart returns -9 for an empty cart, -3 for a Blacklisted customer and -4 for a cart that cannot be checked out

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| `cart_id` | 10 | 12 | 15 | 14 |
| `voucher_id` | *NULL* | *NULL* | *NULL* | *NULL* |
| `payment_method` | Cash | Cash | Cash | Cash |
| `card_id` | *NULL* | *NULL* | *NULL* | *NULL* |
| **Returns** | **-9 (nothing to process)** | **-3 (not allowed)** | **-4 (wrong status)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change | No change | No change |

| Input | Case 5 |
|---|---|
| `cart_id` | 1 |
| `voucher_id` | *NULL* |
| `payment_method` | Cash |
| `card_id` | *NULL* |
| **Returns** | **-4 (wrong status)** |
| **Tables after the call** | No change |

**PU-4.6.i-3** Checkout_Cart returns -7 when the payment cannot be made

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| `cart_id` | 11 | 11 | 11 | 11 |
| `voucher_id` | *NULL* | *NULL* | *NULL* | *NULL* |
| `payment_method` | Wallet | Card | Card | Cheque |
| `card_id` | *NULL* | *NULL* | 1 | *NULL* |
| **Returns** | **-7 (payment cannot be made)** | **-7 (payment cannot be made)** | **-7 (payment cannot be made)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change | No change | No change |

## j) Cancel_Order

**Type:** Stored Procedure **Requirement:** 4.6.j

**PU-4.6.j-1** Cancel_Order cancels everything, refunds to the wallet and restores the voucher

| Input | Case 1 |
|---|---|
| order_id | 8 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Orders | order_id 8 | status | Cancelled |
| SubOrder | suborder_id 12 | status | Cancelled |
| Payment | order_id 8 | refunded_amount | 155.00 |
| Customer | customer_id 22 | wallet_balance | 180.00 |
| Voucher | voucher_id 5 | status | Unused |
| Orders | order_id 8 | voucher_id | NULL |

**PU-4.6.j-2** Cancel_Order returns -4 when a SubOrder was picked up, the order is Cancelled or Delivered

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| order_id | 6 | 4 | 1 |
| **Returns** | **-4 (wrong status)** | **-4 (wrong status)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change | No change |

## k) Reorder

**Type:** Stored Procedure **Requirement:** 4.6.k

**PU-4.6.k-1** Reorder creates a new Active cart with the same items for the chosen address and makes it active

| Input | Case 1 |
|---|---|
| order_id | 1 |
| address_id | 10 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Cart | new cart | status / address_id / customer_id | Active / 10 / 17 |
| CartItem | new cart, product_id 1 | quantity / notes / unit_price | 2 / Extra sauce / NULL |
| CartItem | new cart, product_id 5 | quantity / notes / unit_price | 1 / NULL / NULL |
| Address | address_id 10 | is_active | 1 (the only active address of 17) |

**PU-4.6.k-2** Reorder returns -3 (Blacklisted or not my address), -5 (zone differs) and -2 (address already has an Active cart)

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 | Case 4 |
|---|---|---|---|---|
| order_id | 4 | 1 | 1 | 1 |
| address_id | 7 | 4 | 2 | 1 |
| **Returns** | **-3 (not allowed)** | **-3 (not allowed)** | **-5 (zone mismatch)** | **-2 (duplicate)** |
| **Tables after the call** | No change | No change | No change | No change |

## l) Submit_Feedback

**Type:** Stored Procedure **Requirement:** 4.6.l

**PU-4.6.l-1** Submit_Feedback stores feedback for a Delivered SubOrder

| Input | Case 1 |
|---|---|
| suborder_id | 2 |
| headline | Nice pizza |
| description | Arrived hot |
| rating | 4 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Feedback | suborder_id 2 | headline | Nice pizza |
| Feedback | suborder_id 2 | rating | 4 |

**PU-4.6.l-2** Submit_Feedback returns -2 for a second feedback, -4 for a SubOrder not delivered and -1 for a rating of 6

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| suborder_id | 1 | 9 | 2 |
| headline | Again | Early | Wow |
| description | *NULL* | *NULL* | *NULL* |
| rating | 5 | 5 | 6 |
| **Returns** | **-2 (duplicate)** | **-4 (wrong status)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change | No change |

## m) Rate_Rider

**Type:** Stored Procedure **Requirement:** 4.6.m

**PU-4.6.m-1** Rate_Rider stores the rider rating

| Input | Case 1 |
|---|---|
| suborder_id | 4 |
| rider_rating | 5 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Delivery | suborder_id 4 | rider_rating | 5 |

**PU-4.6.m-2** Rate_Rider returns -2 for a second rating, -4 for a SubOrder not delivered and -1 for a rating of 0

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| `suborder_id` | 1 | 9 | 4 |
| `rider_rating` | 3 | 3 | 0 |
| **Returns** | **-2 (duplicate)** | **-4 (wrong status)** | **-1 (invalid input)** |
| **Tables after the call** | No change | No change | No change |

## n) My_Order_History

**Type:** Table-Valued Function **Requirement:** 4.6.n

**PU-4.6.n** My_Order_History lists all orders of the customer

**Input**

| customer_id |
|---|
| 17 |

**Expected output** (2 rows, in any order)

| order_id | order_date | zone_name | status | number_of_suborders | discount_amount | total_amount | payment_method | refunded_amount |
|---|---|---|---|---|---|---|---|---|
| 1 | 2026-09-01 12:00 | Maadi | Delivered | 2 | 0.00 | 260.00 | Card | 0.00 |
| 6 | 2026-09-20 18:00 | Maadi | In Progress | 2 | 0.00 | 190.00 | Card | 0.00 |

## o) Toggle_Favorite

**Type:** Stored Procedure **Requirement:** 4.6.o

**PU-4.6.o-1** Toggle_Favorite adds and then removes a product favorite

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| `customer_id` | 17 | 17 |
| `item_id` | 2 | 2 |
| `item_type` | Product | Product |
| **Returns** | **0** | **0** |
| **Tables after the call** | See "After case 1" | See "After case 2" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Favorite_Product | customer 17, product 2 | (row) | added |

**After case 2** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Favorite_Product | customer 17, product 2 | (row) | removed |

**PU-4.6.o-2** Toggle_Favorite removes an existing business favorite and returns -1 for an unknown item type

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| `customer_id` | 17 | 17 |
| `item_id` | 1 | 1 |
| `item_type` | Business | Order |
| **Returns** | **0** | **-1 (invalid input)** |
| **Tables after the call** | See "After case 1" | No change |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Favorite_Business | customer 17, business 1 | (row) | removed |

## p) My_Favorites

**Type:** Table-Valued Function **Requirement:** 4.6.p

**PU-4.6.p** My_Favorites lists favorite products only, with is_orderable

**Input**

| customer_id |
|---|
| 17 |

**Expected output** (1 row, in any order)

| product_id | product_name | business_name | is_orderable |
|---|---|---|---|
| 5 | Margherita Pizza | Pizza Roma | 1 |

**Input**

| customer_id |
|---|
| 19 |

**Expected output** (1 row, in any order)

| product_id | product_name | business_name | is_orderable |
|---|---|---|---|
| 15 | Basbousa | Sweet Corner | 0 |

## q) Recommended_Products

**Type:** Table-Valued Function **Requirement:** 4.6.q

**PU-4.6.q** Recommended_Products: unseen products of ordered categories in the active address zone, most ordered category first

**Input**

| customer_id |
|---|
| 17 |

**Expected output** (3 rows, in any order)

| product_id | product_name | business_name | category | final_price |
|---|---|---|---|---|
| 18 | Hawawshi | Koshary El Tahrir | Main Dishes | 50.00 |
| 6 | Pepperoni Pizza | Pizza Roma | Pizza | 150.00 |

| product_id | product_name | business_name | category | final_price |
|---|---|---|---|---|
| 19 | Face Mask | Seif Pharmacy | Hygiene | 15.00 |

## 4.7 Rider

### a) Set_Rider_Availability

**Type:** Stored Procedure **Requirement:** 4.7.a

**PU-4.7.a** Set_Rider_Availability makes a rider available; -3 for a user who is not a rider

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| rider_id | 14 | 17 |
| is_available | 1 | 1 |
| **Returns** | **0** | **-3 (not allowed)** |
| **Tables after the call** | See "After case 1" | No change |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| Rider | rider_id 14 | is_available | 1 |

### b) My_Active_Deliveries

**Type:** Table-Valued Function **Requirement:** 4.7.b

**PU-4.7.b** My_Active_Deliveries lists Assigned and Picked Up deliveries

**Input**

| rider_id |
|---|
| 12 |

**Expected output** (2 rows, in any order)

| delivery_id | order_id | suborder_id | business_name | customer_name | address_description | zone_name | status | assigned_at |
|---|---|---|---|---|---|---|---|---|
| 9 | 6 | 9 | Koshary El Tahrir | Ahmed Salem | Street 9 Building 12 | Maadi | Picked Up | 2026-09-20 18:00 |
| 11 | 7 | 11 | Seif Pharmacy | Sara Mostafa | Road 233 Villa 7 | Maadi | Assigned | 2026-09-21 10:00 |

### c) Mark_Picked_Up

**Type:** Stored Procedure **Requirement:** 4.7.c

**PU-4.7.c-1** Mark_Picked_Up updates the delivery, the SubOrder and the order

| Input | Case 1 |
|---|---|
| delivery_id | 11 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| SubOrder | suborder_id 11 | status | Picked Up |
| Delivery | delivery_id 11 | picked_up_at | now |
| Orders | order_id 7 | status | In Progress |

**PU-4.7.c-2** Mark_Picked_Up returns -4 for Delivered, Cancelled and already Picked Up deliveries

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 | Case 3 |
|---|---|---|---|
| `delivery_id` | 1 | 6 | 9 |
| **Returns** | **-4 (wrong status)** | **-4 (wrong status)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change | No change |

## d) Mark_Delivered

**Type:** Stored Procedure **Requirement:** 4.7.d

**PU-4.7.d-1** Mark_Delivered stamps delivered_at, computes tracking_time and recomputes the order

| Input | Case 1 |
|---|---|
| `delivery_id` | 9 |
| **Returns** | **0** |
| **Tables after the call** | See "After case 1" |

**After case 1** (only these values are checked):

| Table | Row | Column | Expected value |
|---|---|---|---|
| SubOrder | suborder_id 9 | status | Delivered |
| Delivery | delivery_id 9 | delivered_at | now |
| Delivery | delivery_id 9 | tracking_time | minutes from picked_up_at to delivered_at |
| Orders | order_id 6 | status | In Progress (SubOrder 10 is not delivered yet) |

**PU-4.7.d-2** Mark_Delivered returns -4 for a delivery that was not picked up

*Start from the seed data and run the cases from left to right (each case continues from the previous one).*

| Input | Case 1 | Case 2 |
|---|---|---|
| `delivery_id` | 11 | 1 |
| **Returns** | **-4 (wrong status)** | **-4 (wrong status)** |
| **Tables after the call** | No change | No change |

## e) My_Earnings

**Type:** Table-Valued Function **Requirement:** 4.7.e

**PU-4.7.e** My_Earnings for September (80% of the fees of delivered deliveries)

**Input**

| rider_id | from_date | to_date |
|---|---|---|
| 12 | 2026-09-01 | 2026-09-30 |

**Expected output** (1 row, in any order)

| completed_deliveries | total_delivery_fees | total_earnings |
|---|---|---|
| 2 | 40.00 | 32.00 |

## f) My_Rating_Summary

**Type:** Table-Valued Function **Requirement:** 4.7.f

**PU-4.7.f** My_Rating_Summary with star breakdown

**Input**

| rider_id |
|---|
| 12 |

**Expected output** (1 row, in any order)

| avg_rating | ratings_count | five_star | four_star | three_star | two_star | one_star |
|---|---|---|---|---|---|---|
| 4.50 | 2 | 1 | 1 | 0 | 0 | 0 |

## g) My_Delivery_History

**Type:** Table-Valued Function **Requirement:** 4.7.g

**PU-4.7.g** My_Delivery_History lists Delivered and Cancelled deliveries assigned in the range

**Input**

| rider_id | from_date | to_date |
|---|---|---|
| 12 | 2026-09-01 | 2026-09-30 |

**Expected output** (3 rows, in any order)

| delivery_id | order_id | business_name | status | assigned_at | picked_up_at | delivered_at | tracking_time |
|---|---|---|---|---|---|---|---|
| 1 | 1 | Koshary El Tahrir | Delivered | 2026-09-01 12:00 | 2026-09-01 12:10 | 2026-09-01 12:40 | 30 |
| 3 | 2 | Seif Pharmacy | Delivered | 2026-09-02 19:00 | 2026-09-02 19:05 | 2026-09-02 19:30 | 25 |
| 6 | 4 | Pizza Roma | Cancelled | 2026-09-10 20:00 | NULL | NULL | NULL |

## 4.8 System-Level Requirements

### a) Search_Platform

**Type:** Table-Valued Function **Requirement:** 4.8.a

**PU-4.8.a** Search_Platform ranks Active businesses of the customer's active address zone by match type

**Input**

| customer_id | keyword |
|---|---|
| 17 | Pizza |

**Expected output** (1 row, in any order)

| business_id | business_name | priority |
|---|---|---|
| 2 | Pizza Roma | 2 |

**Input**

| customer_id | keyword |
|---|---|
| 17 | Seif Pharmacy |

**Expected output** (1 row, in any order)

| business_id | business_name | priority |
|---|---|---|
| 3 | Seif Pharmacy | 1 |

**Input**

| customer_id | keyword |
|---|---|
| 17 | Pharm |

**Expected output** (1 row, in any order)

| business_id | business_name | priority |
|---|---|---|
| 3 | Seif Pharmacy | 3 |

**Input**

| customer_id | keyword |
|---|---|
| 17 | Sushi |

**Expected output** (no rows)

| business_id | business_name | priority |
|---|---|---|

**Input**

| customer_id | keyword |
|---|---|
| 19 | Pharmacy |

**Expected output** (no rows)

| business_id | business_name | priority |
|---|---|---|

**Input**

| customer_id | keyword |
|---|---|
| 17 | a |

**Expected output** (3 rows, in any order)

| business_id | business_name | priority |
|---|---|---|
| 1 | Koshary El Tahrir | 3 |
| 2 | Pizza Roma | 3 |
| 3 | Seif Pharmacy | 3 |

## b) Reconcile_Order_Totals

**Type:** Table-Valued Function **Requirement:** 4.8.b

**PU-4.8.b** Reconcile_Order_Totals is empty on consistent data and reports tampered orders

**Expected output** (no rows)

| order_id | expected_total | order_total | payment_amount |
|---|---|---|---|

## c) Nearby_Businesses_By_Category

**Type:** Table-Valued Function **Requirement:** 4.8.c

**PU-4.8.c** Nearby_Businesses_By_Category for Desserts near a Maadi address

**Input**

| address_id | product_category |
|---|---|
| 1 | Desserts |

**Expected output** (3 rows, in any order)

| business_id | business_name | zone_name | same_zone | zone_avg_delivery_time |
|---|---|---|---|---|
| 1 | Koshary El Tahrir | Maadi | 1 | 22.50 |
| 2 | Pizza Roma | Maadi | 1 | 22.50 |
| 4 | Sushi Zen | Zamalek | 0 | 35.00 |

## d) Platform_Health_Snapshot

**Type:** Table-Valued Function **Requirement:** 4.8.d

**PU-4.8.d** Platform_Health_Snapshot on the seed data

**Expected output** (1 row, in any order)

| active_businesses | available_riders | orders_today | avg_delivery_time_today | low_ratings_today |
|---|---|---|---|---|
| 5 | 4 | 0 | NULL | 0 |